

Goated U-Fund Design Documentation

Team Information

- Team name: Goated
- Team members
 - Anthony Ficalora
 - Ricardo Lopez
 - Matthew Beicke
 - Zach Coy

Executive Summary

This is a summary of the project.

Purpose

This project aims to enable organizations to have their most important needs satisfied via crowdsourcing. It will allow organization managers to create a list of things that they do need and allow non-managers to contribute to them by adding them to a 'basket' then checking out that basket. Additionally, the project contains security measures such as encrypted passwords, security questions (for forgot password), and API Keys. There is also the permission granted to managers to ban/unban a non manager in the event something requires it. The project also displays statistics on who has contributed the most and what proportion of needs are unfulfilled.

Glossary and Acronyms

Term	Definition
SPA	Single Page Application
User	Anyone who uses the system (either a helper or a manager)
Session	An instance of a user being logged in with an API key.
Helper	Someone who completes needs from the Cupboard.
Manager	Someone who modifies the cupboard and administrates users. Cannot complete Needs.
Cupboard	The full list of uncompleted needs which can be completed, modified, or removed.

Requirements

This section describes the features of the application.

Definition of MVP

Non-profit groups require many donations to be sustainable, and the problem of requesting and satisfying these needs poses a issue. The large, sweeping demands can dissuade potential donors from helping the organization.

What if instead, all of the needs an organization could have were broken down into smaller, more accessible requests? Our project aims to enable prospective supporters to contribute to a greater cause.

Users are able to log in to view the list of available needs. Then, they can select any number of needs to add to their basket, and check out when they are ready.

Additionally, managers can log in to add, edit, and remove needs.

MVP Features

[Sprint 4] *Provide a list of top-level Epics and/or Stories of the MVP.*

Enhancements

Security Package:

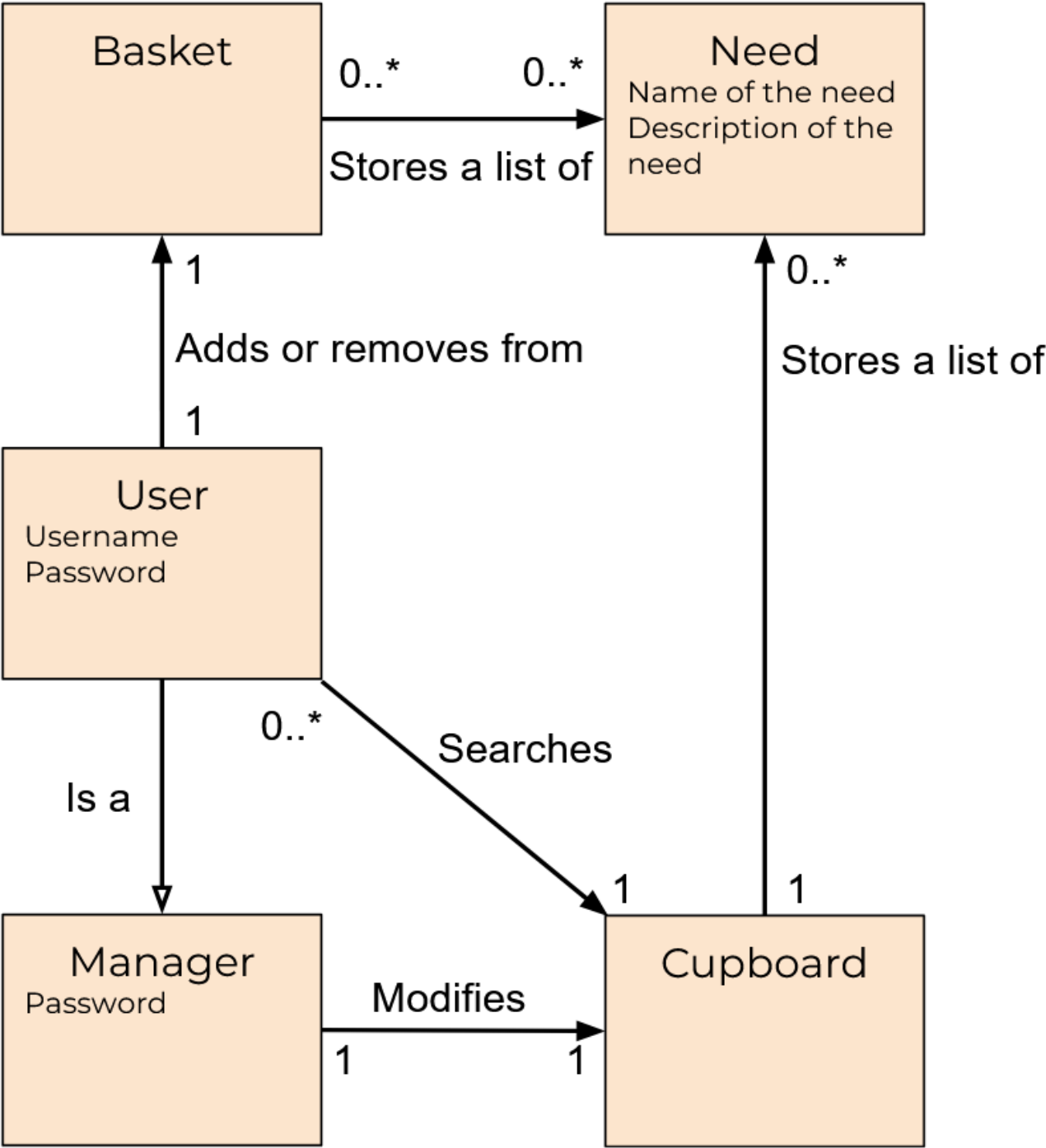
- Passwords are encrypted on the server, and cannot be retrieved after being created.
- API keys are assigned to each user upon login making it impossible for the user to complete some actions without it
- Sessions will expire after one hour
- Managers can Ban and Unban users if they choose to
- All users are required to set a security question that can be answered when they forget their password

Thanks:

- Default page is now the home page that displays the 5 most recently fulfilled needs as well as a button to take you to the login page
- Thanks page displays the list of all needs fulfilled
- On the thanks page there is the number of needs that were fulfilled in the past day, week, month, and year
- Also on the thanks page is a pie chart of fulfilled vs unfulfilled needs that can be sorted by Need name
- Leaderboard tab that displays the users who have contributed the most

Application Domain

This section describes the application domain.



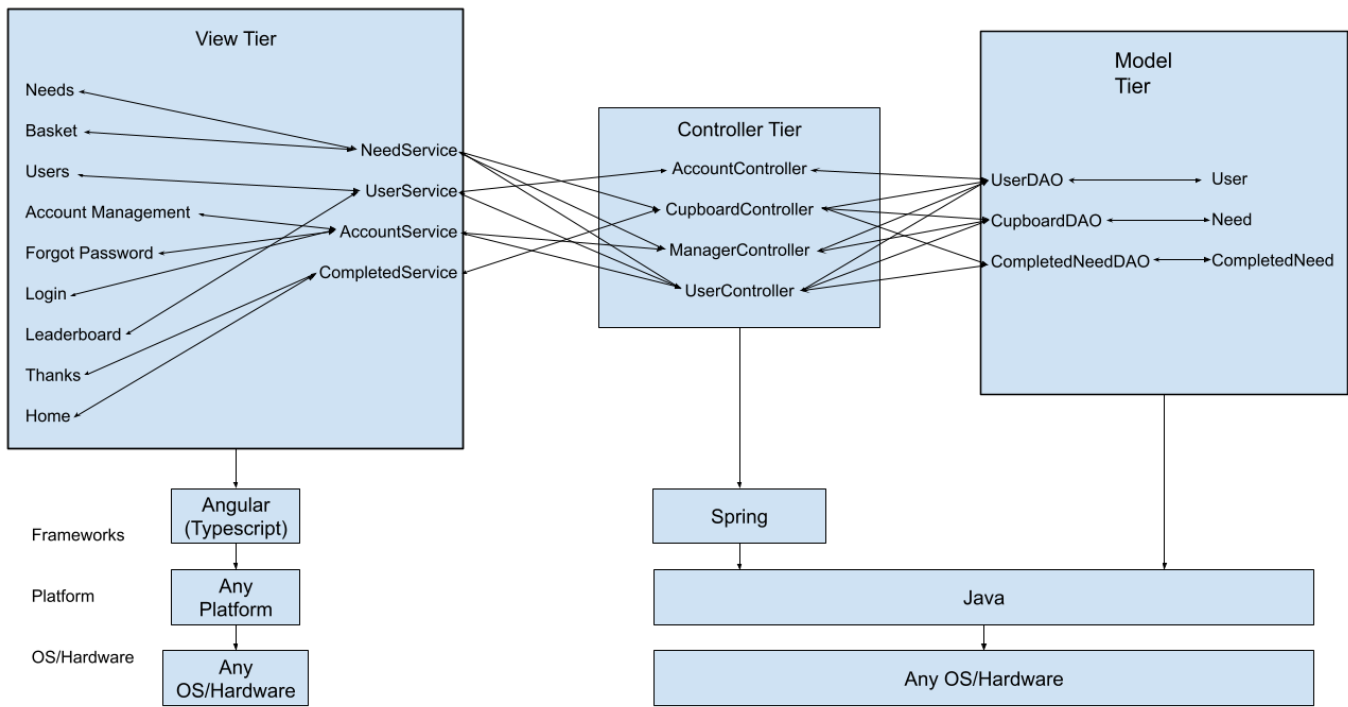
The donation service has managers that can add needs to a public list called a cupboard. Helpers can then choose from the needs added by managers, and add/remove the need to/from a basket that the helper can manage. The helper can then checkout the needs once they've been fulfilled, removing them from both the helper's basket, and the cupboard.

Architecture and Design

This section describes the application architecture.

Summary

The following Tiers/Layers model shows a high-level view of the webapp's architecture.



The application, is built using the Model–View–Controller (MVC) architecture pattern. The Model stores the application data objects including any functionality to provide persistence. The View is the client-side SPA built with Angular utilizing HTML, CSS and TypeScript. The Controller provides RESTful APIs to the client (View) as well as any logic required to manipulate the data objects from the Model. Both the Controller and Model are built using Java and Spring Framework. Details of the components within these tiers are supplied below.

Overview of User Interface

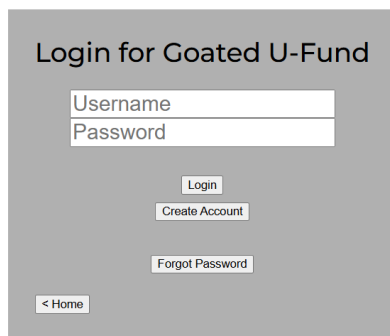
The first page the user sees when visiting the website is the home page. Here they can see the 5 most recently fulfilled needs as well as a button that they can press to take them to the login page.

Thanks to our most recent contributors:

Canned Corn was fulfilled by dave on 11/04/2025 at 7:51 PM.
Oatmeal was fulfilled by Deleted Account on 11/04/2025 at 7:51 PM.
Canned Tuna was fulfilled by Deleted Account on 11/04/2025 at 7:34 PM.
Canned Fruit was fulfilled by Deleted Account on 11/03/2025 at 6:13 PM.
Cereal was fulfilled by Deleted Account on 11/03/2025 at 6:13 PM.

Login

Upon pressing the the login button the users are taken to the login page. Here they can enter their username and password (of which the password appears as only dots) and then press either login to login or Create Account to create an account with the inputted information. If the user forgets their password they can press the 'Forgot Password' button and will be prompted to enter their username and then taken to the forgot password page. Finally, the user can return to the home page by pressing 'Home'



A login form titled "Login for Goated U-Fund". It features two input fields: "Username" and "Password". Below the "Password" field is a "Login" button. Further down are two more buttons: "Create Account" and "Forgot Password". At the bottom left is a button labeled "< Home".

When the user presses the forgot password button on the login page they are taken here, the forgot password page where they first must answer their security question. After getting it correct, they may update their password and are taken back to the login page.

Sample Security Question



A form for a sample security question. It consists of a text input field followed by a "Verify" button.

After successfully logging in or creating an account a user is taken to the cupboard page, seen here being viewed as a helper. Here a help can search for needs and/or add needs to their basket.

Needs

Account Management

Basket

Thank You!

Leaderboard

Log Out

All Needs

Search Needs

Name	Description	
Rice	5 lb bags, white or brown	Add to Basket
Canned Soup	10 cans, low sodium preferred	Add to Basket
Peanut Butter	4 jars, creamy or crunchy	Add to Basket
Macaroni and Cheese	10 boxes, Kraft or store brand	Add to Basket
Spaghetti	6 packs, 1 lb each	Add to Basket
Canned Chili	8 cans, with or without beans	Add to Basket
Granola Bars	4 boxes, assorted flavors	Add to Basket
Shelf-Stable Milk	6 cartons, 1 quart each	Add to Basket
Instant Noodles	12 packs, ramen or similar	Add to Basket
Cooking Oil	3 bottles, vegetable or canola	Add to Basket
Sugar	4 bags, white granulated, 4 lb	Add to Basket
Flour	4 bags, all-purpose, 5 lb	Add to Basket
Salt	2 containers, iodized	Add to Basket

If instead the user is logged in as a manager they will see this page in which, alongside searching for needs, lets them add, edit, and delete needs.

Needs

Account Management

Users

Thank You!

Leaderboard

Log Out

All Needs

Search Needs

Create a new Need

Name	Description	
Rice	5 lb bags, white or brown	Edit/Delete
Canned Soup	10 cans, low sodium preferred	Edit/Delete
Peanut Butter	4 jars, creamy or crunchy	Edit/Delete
Macaroni and Cheese	10 boxes, Kraft or store brand	Edit/Delete
Spaghetti	6 packs, 1 lb each	Edit/Delete
Canned Chili	8 cans, with or without beans	Edit/Delete
Granola Bars	4 boxes, assorted flavors	Edit/Delete
Shelf-Stable Milk	6 cartons, 1 quart each	Edit/Delete
Instant Noodles	12 packs, ramen or similar	Edit/Delete
Cooking Oil	3 bottles, vegetable or canola	Edit/Delete
Sugar	4 bags, white granulated, 4 lb	Edit/Delete
Flour	4 bags, all-purpose, 5 lb	Edit/Delete
Salt	2 containers, iodized	Edit/Delete

If a helper presses on the "Basket" button on the sidebar they can reach the basket page where they can removing things from their basket and/or checkout their basket.

Needs

Account Management

Basket

Thank You!

Leaderboard

Log Out

Your Basket

Checkout Basket

Name	Description	
Rice	5 lb bags, white or brown	Remove From Basket

If a user presses on the "Account Management" button on the sidebar they can reach a page where they can edit their account information such as their password and username as well as delete their account (only as a helper for the latter two)

Needs

Account Management

Basket

Thank You!

Leaderboard

Log Out

Change Username

Change Password

DELETE ACCOUNT

If a user presses on the "Leaderboard" button on the sidebar they can reach the page where they can see the users who have contributed the most to this organization. They can also set the max amount of users they

wish to see

Needs

Account Management

Users

Thank You!

Leaderboard

Log Out

Top 3 Highest Contributors

Username	Contributions
dave	20
matt	15
test	2

If a user presses on the "Thank you" button on the sidebar they can reach the page where they can see a list of all needs fulfilled, including the user who fulfilled it and the timestamp, a pie chart that displays the proportion of needs currently fulfilled, and the number of needs fulfilled in the past day, week, month, and year.

Needs

Account Management

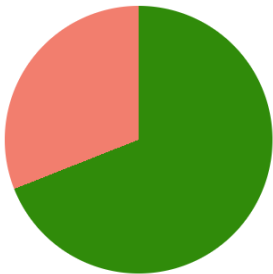
Users

Thank You!

Leaderboard

Log Out

Check completed needs



69% of needs completed!

- Number of needs fulfilled in the past day: 0
- Number of needs fulfilled in the past week: 3
- Number of needs fulfilled in the past month: 96
- Number of needs fulfilled in the past year: 96

<<<

>>>

Recently Satisfied Needs - page 1

- Canned Corn was fulfilled by dave on 11/04/2025 at 7:51 PM. Thank you for your help!
- Oatmeal was fulfilled by Deleted Account on 11/04/2025 at 7:51 PM. Thank you for your help!
- Canned Tuna was fulfilled by Deleted Account on 11/04/2025 at 7:34 PM. Thank you for your help!
- Canned Fruit was fulfilled by Deleted Account on 11/03/2025 at 6:13 PM. Thank you for your help!

If a manager presses on the "Users" tab on the sidebar they can see a list of all users, can search them by their username, and can ban/unban them.



View Tier

[Sprint 4] Provide a summary of the View Tier UI of your architecture. Describe the types of components in the tier and describe their responsibilities. This should be a narrative description, i.e. it has a flow or "story line" that the reader can follow.

[Sprint 4] You must provide at least **2 sequence diagrams** as is relevant to a particular aspects of the design that you are describing. (**For example**, in a shopping experience application you might create a sequence diagram of a customer searching for an item and adding to their cart.) As these can span multiple tiers, be sure to include an relevant HTTP requests from the client-side to the server-side to help illustrate the end-to-end flow.

[Sprint 4] To adequately show your system, you will need to present the **class diagrams** where relevant in your design. Some additional tips:

- Class diagrams only apply to the **Controller** and **Model** Tier
- A single class diagram of the entire system will not be effective. You may start with one, but will be need to break it down into smaller sections to account for requirements of each of the Tier static models below.
- Correct labeling of relationships with proper notation for the relationship type, multiplicities, and navigation information will be important.
- Include other details such as attributes and method signatures that you think are needed to support the level of detail in your discussion.

Controller Tier

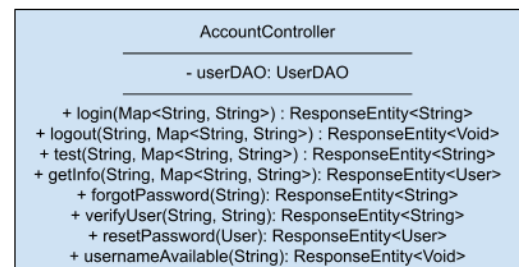
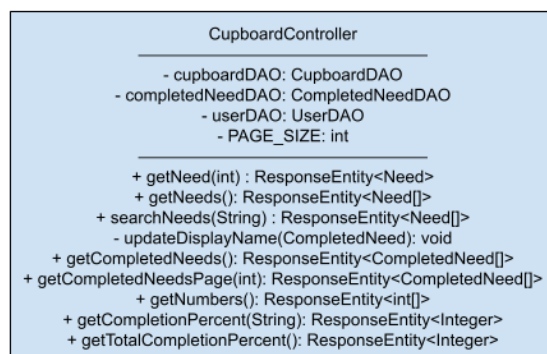
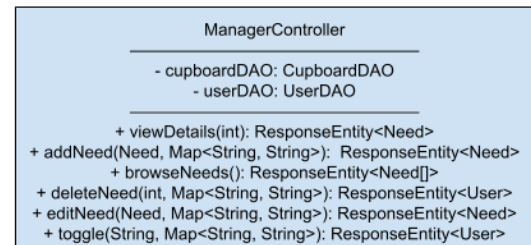
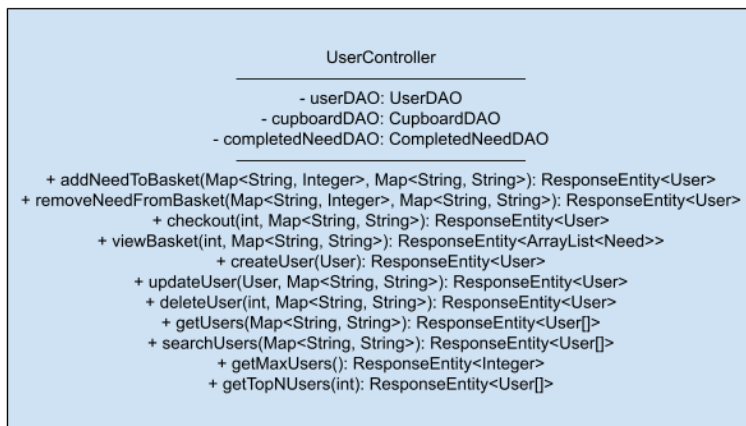
Account Controller - Provides API functionality for login, logout, api key verification, etc

Cupboard Controller - Provides API functionality to access Need and CompletedNeed objects

User Controller - Provides API functionality to for all Helper and mass user related tasks

[Sprint 4] Provide a summary of this tier of your architecture. This section will follow the same instructions that are given for the View Tier above.

At appropriate places as part of this narrative provide **one** or more updated and **properly labeled** static models (UML class diagrams) with some details such as associations (connections) between classes, and critical attributes and methods. (**Be sure** to revisit the Static **UML Review Sheet** to ensure your class diagrams are using correct format and syntax.)



Model Tier

Need: A need is some item that a manager needs funded.

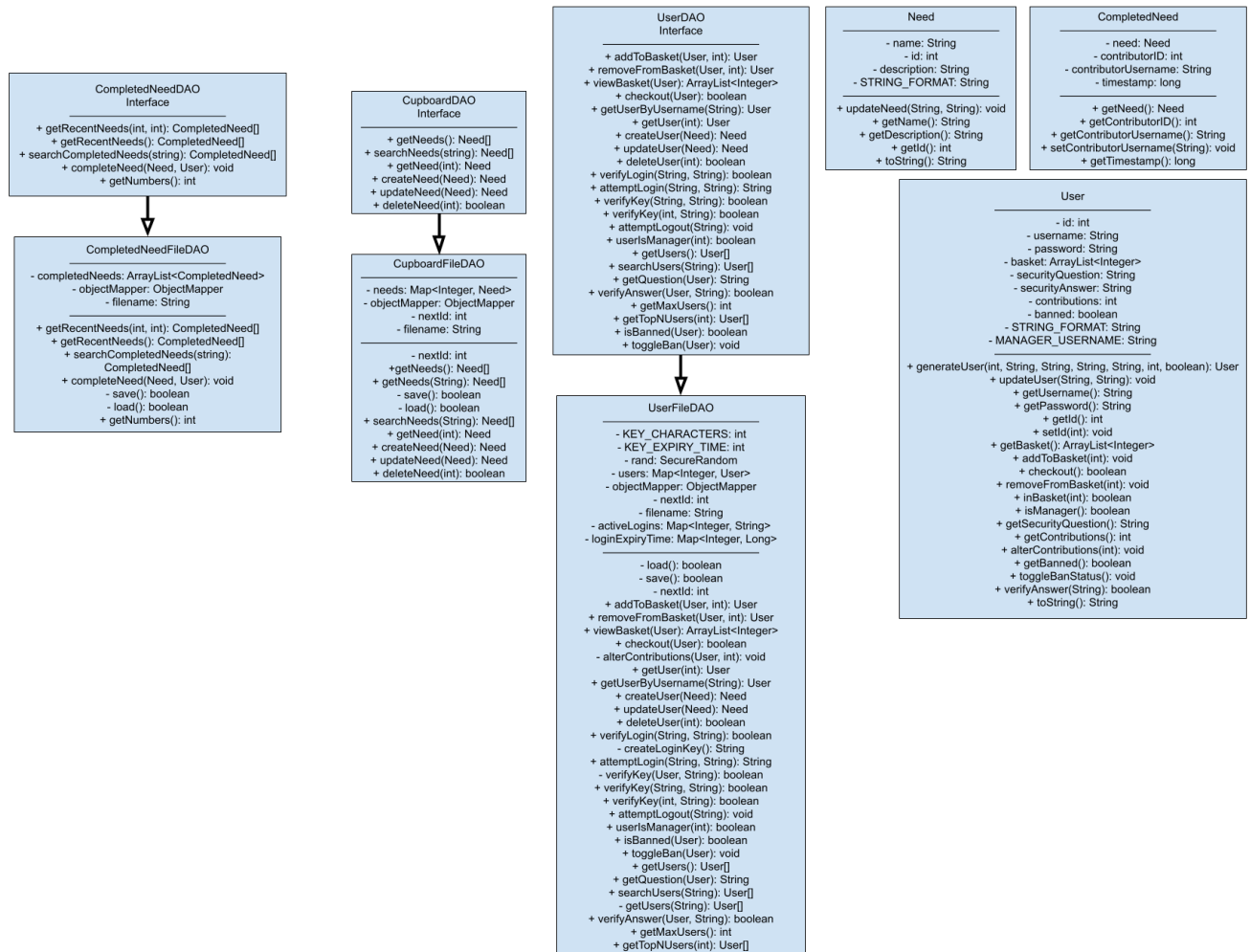
CompletedNeed: A Need that was fulfilled, containing the timestamp of fulfillment and user who did the fulfilling. **User:** Any user, whether it be a Helper or Manager. Both can edit their accounts and have a basket, but Managers cannot modify their own.

UserDAO: The UserDAO provides functions to store and edit users.

CupboardDAO: The CupboardDAO provides functions to store and edit the cupboard.

CompletedNeedDAO: Provides functions to store and edit the completedneed catalog.

In this tier, interaction with the raw data is done and manipulated. The methods in the classes here are used in the Controllers to accomplish their goals.



OO Design Principles

Design Principles in our code:

- **Single Responsibility:** The User, Need, and CompletedNeed objects have their own Controllers. Along with that, the Need and User objects only interact with themselves, and will not attempt to alter any other object. Each of the controllers only do what they alone need to do, such as UserController *only* determining which DAOs will be activated, and UserDao *only* modifying the User objects that need to be edited, and not modifying any other objects (i.e. completing a Need, which is CupboardDAO's responsibility)
- **Open/Closed:** We created DAO (e.g. CupboardDAO) interfaces and FileDAO (e.g. CupboardFileDAO) classes. Of these, CupboardDAO is closed for modification since exactly all of its functionality must be implemented, while CupboardFileDAO is open to include more features such as internal functions needed to save data to disk. This can be clearly seen in the Model Tier diagram, where UserFileDAO has many more functions than UserDao had.
- **Dependency Injection:** The various Controllers take as input the interface DAO, instead of the FileDAO implementation. On the web side, each of the components take in Services in their constructors meaning they can be easily swapped out for another implementation.
- **Controller:** The User, Need, and CompletedNeed objects all have their own Controllers, which lie as one of the intermediate steps between the user and the backend storage. This can be seen in the Tiers/Layers model where each of the controllers lie between front and backend.

- Information Expert: Each class has its own DAO file and cannot access the other DAO files. Additionally, each piece of necessary information is retrieved from its storage medium instead of being taken from another controller or being haphazardly stored by itself.
- Low Coupling: Low coupling was difficult to achieve as many functions demanded access to multiple files at once, such as generating the strings for completed needs, requiring a completed need object and a user object, which would otherwise be unnecessary in the CupboardController. However, classes in the same layer were decoupled to not depend on each other - a Controller only ever called a DAO, which then only ever modified some Model.

Static Code Analysis/Future Design Improvements

[Sprint 4] With the results from the Static Code Analysis exercise, **Identify 3-4** areas within your code that have been flagged by the Static Code Analysis Tool (SonarQube) and provide your analysis and recommendations.

Include any relevant screenshot(s) with each area.

[Sprint 4] Discuss **future** refactoring and other design improvements your team would explore if the team had additional time.

Testing

Acceptance Testing

By the end of Sprint 3 we have 60 User stories.

Currently, for the acceptance criteria we have, all stories pass.

The only issues that would arise were from faulty code. These would be fixed during the testing phase when another team member would analyze their code and figure out what went wrong, collaborate with the creator, and fix it.

Unit Testing and Code Coverage

Our strategy for unit testing was ensure everything is covered, all possible cases. Currently (end of Sprint 3) our code coverage is the following:

ufund-api

ufund-api

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.ufund.api.ufundapi.controller		96%		94%	6	122	8	367	0	55	0	4
com.ufund.api.ufundapi		0%	n/a		4	4	7	7	4	4	2	2
com.ufund.api.ufundapi.persistence		100%		95%	4	97	0	246	0	51	0	3
com.ufund.api.ufundapi.model		100%		100%	0	40	0	70	0	33	0	3
Total	97 of 3,184	96%	12 of 240	95%	14	263	15	690	4	143	2	12

Ongoing Rationale

2025/10/19: Sprint 2

Main programming for Sprint 2 is now complete, merged, and mostly tested (still need to do the actual acceptance testing writeup but).

2025/10/21: Sprint 2

All programming tasks, testing, documentation, demo planning has been completed at this time.

2025/11/9: Sprint 3

All programming for this sprint has been completed at this time, just documentation, demo, and acceptance testing left

2025/11/12: Sprint 3

After these docs are pushed all code for the project should be completed (Sprint 3 is done).